

# Discovery Executive Summary

Project: discovery-16july · Generated: 16/07/2026, 14:24:14

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 4 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

## Portfolio Overview

| # | Analysis                           | Overall Rating | Hotspot Score       |
|---|------------------------------------|----------------|---------------------|
| 1 | Architecture & Design Analysis     | HIGH RISK      | —                   |
| 2 | Code Quality & Complexity Analysis | HIGH RISK      | 46 / 100 — Moderate |
| 3 | Frontend Modernization Analysis    | HIGH RISK      | —                   |
| 4 | Backend Modernization Analysis     | HIGH RISK      | —                   |

## 1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk domain boundary violations (H8), shared-database coupling (H9), duplicated cross-layer logic (H10), and parallel Laravel/dev-api stacks (H11).

EXECUTIVE SUMMARY

FSDKC (Klearcom voice observability platform) spans three runtimes: a Laravel 11 REST API (`backend/`, 26 PHP files), a React 18 SPA (`frontend/src/`, 15 TS/JS files), and a parallel Express dev-api (`dev-api/`, 9 JS files). Controllers are thin by LOC (avg 65), but business workflows are split inconsistently — reachability math and IVR tree building are copy-pasted across controllers, services, and dev-api. There are zero repository classes; all persistence goes through Eloquent models directly from controllers. The dominant risks are **cross-domain coupling** (Dashboard and LegacyReport controllers query both Connect and Discovery models in one schema), **shared-database coupling** (80% of business tables accessed across domains), and **parallel-stack duplication** (Laravel + dev-api mirror the same realtime/test logic). Frontend

architecture is healthier (shared `api/client.ts`, Zustand, React Query hooks) but pages still embed query/mutation wiring inline and one legacy class component remains.

## 1.1 Benchmark Ratings Summary

| #  | Hotspot                             | Primary KPI                        | GOOD | MODERATE | HIGH RISK | Measured                                    | Rating    |
|----|-------------------------------------|------------------------------------|------|----------|-----------|---------------------------------------------|-----------|
| H1 | Fat Controllers                     | Avg LOC per controller             | <150 | 150–300  | >300      | 65 LOC (6 API controllers)                  | GOOD      |
| H2 | Missing Service Layer               | Controllers accessing repos/models | <10  | 10–20    | >20       | 4 controllers                               | GOOD      |
| H3 | Missing Repository Pattern          | Direct DB access points            | <10  | 10–20    | >20       | 6 files                                     | GOOD      |
| H4 | Circular Dependencies               | Dependency cycles                  | 0    | 1–3      | >3        | 2 Eloquent relation pairs                   | MODERATE  |
| H5 | Shared Utility Abuse                | Utility files w/ business logic    | 0    | 1–5      | >5        | 1 file<br>( <code>LegacyDataMapper</code> ) | MODERATE  |
| H6 | Direct SQL in Controllers           | ORM compliance %                   | >90% | 60–90%   | <60%      | 100% Eloquent                               | GOOD      |
| H7 | God Classes                         | Classes >1000 LOC                  | 0    | 1–3      | >3        | 0                                           | GOOD      |
| H8 | Domain Boundary Violations          | Cross-domain access points         | 0    | 1–5      | >5        | 8                                           | HIGH RISK |
| H9 | Shared Database Coupling            | Tables shared across domains       | <10% | 10–30%   | >30%      | 80% (4/5 tables)                            | HIGH RISK |
| F1 | Business Logic in Components        | Avg LOC per component              | <150 | 150–300  | >300      | 75 LOC (11 components/pages)                | GOOD      |
| F2 | Missing Frontend Service/Data Layer | Components w/ inline API calls     | <10  | 10–20    | >20       | 5 pages/widgets                             | GOOD      |
| F3 | God / Oversized Components          | Components >400 LOC                | 0    | 1–3      | >3        | 0                                           | GOOD      |
| F4 | Prop Drilling / Global State Abuse  | Max prop-drilling depth            | ≤2   | 3–4      | >4        | 2 levels                                    | GOOD      |

|     |                                          |                                    |   |      |     |                               |           |
|-----|------------------------------------------|------------------------------------|---|------|-----|-------------------------------|-----------|
| F5  | Legacy / Inconsistent Component Patterns | Legacy-pattern components          | 0 | 1–10 | >10 | 1 class + 1 <code>.jsx</code> | MODERATE  |
| H10 | Duplicated Domain Logic (additional)     | Copy-pasted workflow sites         | 0 | 1–3  | >3  | 6 sites                       | HIGH RISK |
| H11 | Parallel Runtime Stacks (additional)     | Full duplicate API implementations | 0 | 1    | >1  | 2 (Laravel + dev-api)         | HIGH RISK |

## 1.4 Actions Required

| Hotspot | Action                                                                                                                         | Rating    | Priority |
|---------|--------------------------------------------------------------------------------------------------------------------------------|-----------|----------|
| H4      | Encapsulate bidirectional Eloquent relations inside context-specific repositories; expose DTOs at boundaries                   | MODERATE  | MEDIUM   |
| H5      | Replace <code>extract()</code> in <code>LegacyDataMapper</code> and <code>LegacyReportController</code> with typed DTO mappers | MODERATE  | MEDIUM   |
| H8      | Split cross-domain controllers/services; introduce ACL between Connect and Discovery contexts                                  | HIGH RISK | CRITICAL |
| H9      | Assign table ownership per domain; replace cross-domain Dashboard queries with read-model projections                          | HIGH RISK | HIGH     |
| F5      | Migrate <code>LegacyMonitorPoller</code> to function component; add Error Boundary in <code>App.tsx</code>                     | MODERATE  | MEDIUM   |
| H10     | Extract <code>ReachabilityCalculator</code> and <code>IvrTreeBuilder</code> ; delete 6 duplicate implementations               | HIGH RISK | CRITICAL |
| H11     | Deprecate or proxy <code>dev-api/</code> ; designate Laravel as single API source of truth                                     | HIGH RISK | HIGH     |

## 1.5 Expected Outcomes

- Consolidating reachability and tree-building logic into domain services eliminates KPI drift between Connect checks, legacy reports, and the dev-api simulator.
- Repository interfaces and application services make controllers thin HTTP adapters testable with mocked persistence, raising confidence for schema and MongoDB changes.
- Bounded contexts with an anti-corruption layer let Connect and Discovery evolve independently — including future extraction to separate deployables.
- Retiring the parallel dev-api stack removes dual-maintenance burden and ensures local development matches production Laravel behavior.
- Frontend domain service modules and migrated legacy components produce consistent React patterns with proper cleanup and error isolation.

Full report saved to `docs/discovery/01-architecture-design.md` (orchestration UI will convert to PDF).  
Pipeline artifact copy: `agent-runs/20260716T110541_bql9tn/01-architecture-design.md` .  
Analysis covered **backend** (26 PHP files), **frontend** (15 TS/JS files), and **dev-api** (9 JS files) from `shende-shweta/FSDKC` via GitHub REST API.

## 2. Code Quality & Complexity Analysis

OVERALL CODEBASE RATING — CODE QUALITY & COMPLEXITY

High Risk

Driven by H1 cyclomatic complexity (36 in ConnectPage), H3 oversized ConnectPage component (201 LOC), and H4 cross-runtime business-logic duplication (~11%).

EXECUTIVE SUMMARY

Analysis covered **50 source files** across three layers: **17 frontend** (898 LOC), **26 backend** (918 LOC), and **6 dev-api** (720 LOC), totaling **2,598 LOC** of application code. No stack-specific complexity linter (ESLint `complexity`, PHPMD, Sonar) is configured; metrics were derived via manual branch counting and LOC analysis against GitHub `main`. The codebase is young (**3 commits**) with low churn and clear single-author ownership, but **structural complexity and duplication are elevated**: `ConnectPage.tsx` registers **cyclomatic complexity ~36** and **201 LOC** in a single component, and **parallel Laravel + Express implementations** duplicate realtime test workflows and KPI logic (~11% estimated business-rule duplication). Overall health is **High Risk**, driven by frontend page complexity and cross-runtime business-logic duplication despite favorable churn and ownership signals.

### 2.1 Benchmark Ratings Summary

| #  | Hotspot                    | Primary KPI                 | GOOD | MODERATE | HIGH RISK | Measured                                      | Rating   |
|----|----------------------------|-----------------------------|------|----------|-----------|-----------------------------------------------|----------|
| H1 | High Cyclomatic Complexity | Max complexity per method   | <10  | 10–20    | >20       | 36 ( <code>ConnectPage.tsx</code> component)  | HIGH     |
| H2 | Large Classes              | Largest class LOC           | <300 | 300–1000 | >1000     | 224 ( <code>dev-api/src/server.js</code> )    | GOOD     |
| H3 | Large Functions            | Largest function LOC        | <50  | 50–200   | >200      | 201 ( <code>ConnectPage.tsx</code> component) | HIGH     |
| H4 | Business Logic Duplication | Duplicated business logic % | <5%  | 5–10%    | >10%      | ~11% (est. ~285 LOC duplicated / 2,598 total) | HIGH     |
| H5 | Duplicate Code             | Overall duplicate code      | <5%  | 5–10%    | >10%      | ~8% (structural + block duplicates)           | MODERATE |

|     | (general)                              | %                                 |      |           |                |                                                          |          |
|-----|----------------------------------------|-----------------------------------|------|-----------|----------------|----------------------------------------------------------|----------|
| H6  | High Churn Areas                       | Monthly changes (top files)       | <5   | 5–10      | >10            | 2 (max per file, 6-month window)                         | GOOD     |
| H7  | Defect-Prone Files                     | Fix commits (hottest file)        | 1–3  | 4–5       | >5             | 1 ( <code>frontend/src/App.tsx</code> , logo fix)        | GOOD     |
| H8  | Ownership Issues                       | Top-author ownership %            | >80% | 60–80%    | <60%           | 100% (single author <code>ksabai-gl</code> on hot files) | GOOD     |
| H9  | Dual API Runtimes (additional)         | Parallel endpoint implementations | 0    | 1 runtime | 2+ full stacks | 2 (Laravel + Express)                                    | HIGH     |
| H10 | Missing Lifecycle Cleanup (additional) | Components with interval/SSE leak | 0    | 1         | 2+             | 1 ( <code>LegacyMonitorPoller.jsx</code> )               | MODERATE |

### Hotspot Score breakdown

| Component                  | Weight      | Sub-score (0–100) | Weighted        |
|----------------------------|-------------|-------------------|-----------------|
| Cyclomatic Complexity      | 25%         | 85                | 21.25           |
| Code Churn                 | 25%         | 10                | 2.50            |
| Defect Density             | 20%         | 15                | 3.00            |
| Class/Function Size        | 15%         | 72                | 10.80           |
| Business Logic Duplication | 10%         | 78                | 7.80            |
| Developer Ownership Risk   | 5%          | 5                 | 0.25            |
| <b>Hotspot Score</b>       | <b>100%</b> |                   | <b>46 / 100</b> |

## 2.5 Actions Required

| Hotspot                       | Action                                                                                                                                              | Rating    | Priority |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-----------|----------|
| H1 High Cyclomatic Complexity | Split <code>ConnectPage.tsx</code> and <code>DiscoveryPage.tsx</code> into sub-components; extract query hooks; target CC <10 per unit              | HIGH RISK | CRITICAL |
| H3 Large Functions            | Decompose 201-line <code>ConnectPage</code> into 4 feature components; move <code>getSeedDocuments</code> fixtures to JSON files                    | HIGH RISK | HIGH     |
| H4 Business Logic Duplication | Consolidate reachability formula, <code>buildTree</code> , and test-runner workflows into shared domain services; deprecate duplicate Express logic | HIGH RISK | CRITICAL |
| H5 Duplicate Code (general)   | Extract <code>TreeBuilder</code> utility; add jscpd/PHPCPD to CI with <5% threshold                                                                 | MODERATE  | MEDIUM   |

|                               |                                                                                                                                       |           |        |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-----------|--------|
| H9 Dual API Runtimes          | Designate single production API runtime; document dev-api deprecation or generate from OpenAPI                                        | HIGH RISK | HIGH   |
| H10 Missing Lifecycle Cleanup | Add <code>componentWillUnmount</code> to <code>LegacyMonitorPoller.jsx</code> or migrate to hooks with <code>useEffect</code> cleanup | MODERATE  | MEDIUM |

## 2.6 Expected Outcomes

- Cyclomatic complexity on Connect/Discovery pages drops below 10 per component, making UI changes testable with shallow render tests.
- Business-rule changes (reachability threshold, alert logic) require a single edit in a domain service instead of three coordinated copies.
- Eliminating the dual-runtime pattern halves the API maintenance surface and removes drift between Laravel and Express responses.
- Extracted page sub-components enable Storybook documentation and faster code reviews (<80 LOC per PR file).
- Adding complexity and duplication lint rules to CI prevents regression of hotspots identified in this audit.

## 3. Frontend Modernization Analysis

### OVERALL CODEBASE RATING — FRONTEND MODERNIZATION

## High Risk

Driven by H6 (0% Redux Toolkit adoption) and H7 (0% accessibility attribute coverage) — both additional hotspots rated High Risk.

### EXECUTIVE SUMMARY

The Travelsdin social-network frontend is a React 18.2 application built with Create React App, using functional components and hooks throughout — no legacy class-based components were found. However, the codebase relies on hand-written Redux (actions/reducers/thunks) rather than Redux Toolkit, and nearly half of all view files read from the global Redux store directly. Several preview components duplicate the same user-loading and reaction-toggle patterns, and the messaging feature passes eight callback props through four component layers. Two additional risks stand out: zero accessibility attributes (`aria-*` / `role`) across all 59 scanned components, and direct `document` / `window` DOM manipulation inside React views. Overall modernization health is **High Risk**, driven primarily by legacy state-management patterns and missing accessibility infrastructure.

## 3.1 Benchmark Ratings Summary

| # | Hotspot | Primary KPI | GOOD | MODERATE | HIGH RISK | Measured | Rating |
|---|---------|-------------|------|----------|-----------|----------|--------|
|---|---------|-------------|------|----------|-----------|----------|--------|

|    |                                               |                                           |         |           |          |                                      |           |
|----|-----------------------------------------------|-------------------------------------------|---------|-----------|----------|--------------------------------------|-----------|
| H1 | UI Component Duplication                      | Duplicate components %                    | <5%     | 5–10%     | >10%     | 8.5% (5 of 59)                       | MODERATE  |
| H2 | Legacy Class-Based Components                 | Modern component adoption %               | >90%    | 70–90%    | <70%     | 100% (59/59 functional)              | GOOD      |
| H3 | Massive Components                            | Largest component LOC                     | <200    | 200–500   | >500     | 250 LOC ( <code>Message.jsx</code> ) | MODERATE  |
| H4 | Global State Dependencies                     | Components reading global state %         | <30%    | 30–60%    | >60%     | 49% (29/59)                          | MODERATE  |
| H5 | Complex State Management                      | Max prop-drilling depth                   | <3      | 3–5       | >5       | 4 levels (Message → MsgPreview)      | MODERATE  |
| H6 | Legacy Redux Boilerplate (additional)         | Redux Toolkit adoption %                  | >80%    | 50–80%    | <50%     | 0% (0/4 modules)                     | HIGH RISK |
| H7 | Missing Accessibility Attributes (additional) | Components with aria/role coverage %      | >50%    | 20–50%    | <20%     | 0% (0/59)                            | HIGH RISK |
| H8 | Direct DOM Manipulation (additional)          | Components using document/window DOM APIs | 0 files | 1–3 files | >3 files | 3 files                              | MODERATE  |

### 3.5 Actions Required

| Hotspot                        | Action                                                                                                                                    | Rating    | Priority |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------|----------|
| H6 — Legacy Redux Boilerplate  | Add <code>@reduxjs/toolkit</code> ; migrate all 4 reducers to <code>createSlice</code> + <code>configureStore</code>                      | HIGH RISK | CRITICAL |
| H7 — Missing Accessibility     | Add <code>aria-label</code> , <code>role</code> , and meaningful <code>alt</code> text to all interactive elements across 59 components   | HIGH RISK | HIGH     |
| H1 — UI Component Duplication  | Extract <code>useUserProfile</code> , <code>useReactionToggle</code> hooks and <code>UserAvatarCard</code> shared component               | MODERATE  | MEDIUM   |
| H3 — Massive Components        | Split <code>Message.jsx</code> hook, <code>CreatePostModal</code> , and <code>CommentPreview</code> into sub-components                   | MODERATE  | MEDIUM   |
| H4 — Global State Dependencies | Introduce RTK typed selectors and <code>AuthContext</code> to reduce 49% direct store reads                                               | MODERATE  | MEDIUM   |
| H5 — Complex State Management  | Create <code>ChatContext</code> provider to eliminate 4-level messaging prop chain                                                        | MODERATE  | MEDIUM   |
| H8 — Direct DOM Manipulation   | Replace <code>document.querySelector</code> and <code>window</code> scroll with <code>useRef</code> and <code>IntersectionObserver</code> | MODERATE  | MEDIUM   |

### 3.6 Expected Outcomes

- Migrating to Redux Toolkit eliminates ~200 lines of boilerplate action/reducer code and provides Immer-safe immutable updates, reducing mutation bugs in like/comment flows.
- A shared `UserAvatarCard` and `useReactionToggle` hook consolidates 5 duplicate preview components, cutting repeated bug-fix surface by ~8.5%.
- `ChatContext` and `AuthContext` providers will reduce the 49% global-store coupling, making components testable without a full Redux mock.
- Adding WCAG-compliant ARIA attributes enables screen-reader and keyboard access for messaging, notifications, and post interactions.
- Replacing direct DOM calls with `useRef` and `IntersectionObserver` makes scroll behavior testable and compatible with future SSR or React 19 concurrent rendering.

### 4. Backend Modernization Analysis

OVERALL CODEBASE RATING — BACKEND MODERNIZATION

High Risk

Driven by H3 (21.9% data-layer compliance), H5 (36 inline handlers), H6 (15% single-API capabilities), H7 (0% API governance), and H8 (duplicated cross-runtime logic).

EXECUTIVE SUMMARY

The FSDKC (Klearcom) backend is a dual-runtime stack: a Laravel 12 monolith (`backend/`) backed by Eloquent/MariaDB and MongoDB, plus a parallel Express dev-api (`dev-api/`) that re-implements most REST capabilities against an in-memory store. Architectural layering is immature — only two injectable service classes exist, Eloquent and KPI math live directly in controllers, and 17 of 20 API capabilities are duplicated across both runtimes with no OpenAPI spec, versioning, or contract tests. PHP `extract()` on user input appears in legacy reporting code, and the dev-api exposes unvalidated `req.body` writes plus module-level mutable globals. Overall backend modernization posture is **High Risk**, driven by missing data/service layers, API sprawl, absent API governance, and duplicated business logic across PHP and Node.

#### 4.1 Benchmark Ratings Summary

| #  | Hotspot                   | Primary KPI                        | GOOD | MODERATE | HIGH RISK | Measured                                                                      | Rating   |
|----|---------------------------|------------------------------------|------|----------|-----------|-------------------------------------------------------------------------------|----------|
| H1 | Dynamic Variable Creation | Dynamic-var-from-input occurrences | 0    | 1–10     | >10       | 3 <code>extract()</code> + unchecked <code>req.body</code> in 4 POST handlers | MODERATE |



|    |                                             |                                                 |      |        |      |                                                                                                                                                           |          |
|----|---------------------------------------------|-------------------------------------------------|------|--------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| H2 | Global Mutable State                        | Globals / mutable static state                  | 0    | 1–5    | >5   | 2 modules ( <code>store.js</code> , <code>mongo.js</code> )                                                                                               | MODERATE |
| H3 | Direct SQL Outside Data Layer               | Data-layer compliance %                         | >90% | 60–90% | <60% | 21.9% (7 of 32 Eloquent calls in services)                                                                                                                | HIGH     |
| H4 | Static / Singleton Abuse                    | Business-logic static/singleton classes         | 0    | 1–5    | >5   | 0                                                                                                                                                         | GOOD     |
| H5 | Missing Service Layer                       | Handlers with inline business logic             | <10  | 10–20  | >20  | 36 (18 Laravel methods + 18 Express routes)                                                                                                               | HIGH     |
| H6 | API Sprawl                                  | Documented & governed endpoints %               | >90% | 80–90% | <80% | 15% single-API capabilities (3 of 20)                                                                                                                     | HIGH     |
| H7 | Missing API Governance                      | Governance compliance %                         | 100% | 90–99% | <90% | 0% (no OpenAPI, versioning, or contract tests)                                                                                                            | HIGH     |
| H8 | Duplicated Cross-Runtime Logic (additional) | Duplicate business-logic blocks across PHP/Node | 0    | 1–3    | >3   | 6 (reachability ×3, <code>buildTree</code> ×3)                                                                                                            | HIGH     |
| H9 | Unvalidated dev-api Inputs (additional)     | POST/PUT handlers without schema validation     | 0    | 1–3    | >3   | 4 ( <code>/discovery/jobs</code> , <code>/discovery/jobs/:id/start</code> , <code>/connect/monitors</code> , <code>/connect/monitors/bulk-import</code> ) | HIGH     |

## 4.5 Actions Required

| Hotspot                                        | Action                                                                                                                                          | Rating    | Priority |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------|----------|
| H3 Direct SQL Outside Data Layer               | Create Repository classes for Connect, Discovery, and CheckResult models; move all Eloquent chains out of controllers                           | HIGH RISK | CRITICAL |
| H5 Missing Service Layer                       | Add DashboardService, ConnectService, DiscoveryService, LegacyReportService; slim controllers to delegation only                                | HIGH RISK | CRITICAL |
| H6 API Sprawl                                  | Deprecate duplicate dev-api routes; document Laravel as canonical API; remove bulk-import shim or port to Laravel with validation               | HIGH RISK | HIGH     |
| H7 Missing API Governance                      | Publish OpenAPI 3 spec, add <code>/api/v1/</code> versioning, Spectral lint, and PHPUnit contract tests                                         | HIGH RISK | HIGH     |
| H8 Duplicated Cross-Runtime Logic (additional) | Extract shared reachability + tree-building into single PHP services; delete mirrored logic in <code>realtime.js</code> / <code>store.js</code> | HIGH RISK | HIGH     |
| H9 Unvalidated dev-api Inputs (additional)     | Add Joi/Zod schemas for all dev-api POST/PUT routes; block bulk-import until validated                                                          | HIGH RISK | HIGH     |

|                              |                                                                                                                     |          |        |
|------------------------------|---------------------------------------------------------------------------------------------------------------------|----------|--------|
| H1 Dynamic Variable Creation | Remove all <code>extract()</code> usage; replace with typed DTOs in LegacyDataMapper and LegacyReportController     | MODERATE | HIGH   |
| H2 Global Mutable State      | Replace <code>store.js</code> singleton with injectable repository; encapsulate Mongo globals in connection manager | MODERATE | MEDIUM |

## 4.6 Expected Outcomes

- Repository + service layers raise data-layer compliance above 90% and make domain logic testable without HTTP bootstrapping.
- Retiring duplicate dev-api routes eliminates API sprawl and gives integrators one canonical contract per capability.
- OpenAPI specs with contract tests catch breaking response changes before they reach the React frontend.
- Removing `extract()` and adding DTO validation closes untyped data-flow gaps in legacy reporting and dev-api writes.
- Consolidating reachability and tree-building logic prevents KPI drift between PHP and Node runtimes.

Full report saved to <target/docs/discovery/04-backend-modernization.md> (PDF conversion runs automatically in the orchestration UI).